

Finishline Infra Project Assignment – Part 2

Deploy Karpenter to you EKS cluster to manage your instance

Reporter: Joseph Ndzoh Dong

Start date: Monday March 16th, 2026

End date: Thursday March 19th, 2026

Team Lead: Ferdinand

Assignee: Finish Line Seniors

Objective

Karpenter helps your cluster add servers when workloads need more capacity and reduce servers when capacity is wasted. It does this by evaluating pod requirements such as CPU, memory, node selectors, affinities, tolerations, and topology rules, then creating matching nodes through the cloud provider.

Scenario

Deploy **Karpenter v1.8** on the existing **EKS** cluster using the current **v1 APIs** (NodePool and EC2NodeClass). Install Karpenter with Helm and configure it so that it provisions only **t3a** instances and uses **Bottlerocket** as the AMI family. Karpenter's EKS install flow, Helm install, and v1 resource model are documented in the official getting-started and migration docs, and Bottlerocket is configured through **amiFamily** in EC2NodeClass.

1. Confirm the EKS cluster is already running, and that cluster access works with kubectl, aws, helm, and eksctl. Karpenter's official install flow assumes those tools and an existing supported EKS environment.
2. Create or verify the AWS permissions required for Karpenter:
 - IAM role for the **Karpenter controller** using **IRSA**
 - IAM role/profile for the **EC2 nodes** Karpenter will launch
Karpenter requires cloud-provider credentials through IRSA to describe instance types and launch EC2 capacity.
3. Install the **Karpenter CRDs** first with Helm.
4. Install the **Karpenter controller** in the cluster with Helm in kube-system.
5. Create an **EC2NodeClass** that uses:
 - amiFamily: Bottlerocket
 - the correct node IAM role
 - subnet selectors for the cluster
 - security group selectors for the cluster
6. Create a **NodePool** that restricts provisioning to:
 - kubernetes.io/os=linux
 - kubernetes.io/arch=amd64
 - node.kubernetes.io/instance-type set to only **t3a** instance types, such as:

- t3a.medium
Karpenter supports selecting instance types by label requirements in the NodePool/NodeClaim flow.

7. Apply both manifests:
 - ec2nodeclass.yaml
 - nodepool.yaml
8. Deploy a test workload with enough CPU and memory requests to force unschedulable pods so Karpenter must provision new nodes. Karpenter creates **NodeClaims** in response to pending pods and
9. Verify the deployment by checking:
 - Karpenter controller pods are running
 - the NodePool is created
 - NodeClaims are created
 - new worker nodes join the cluster
 - the new nodes are **t3a** instances
 - the new nodes use **Bottlerocket**

The core runtime objects to monitor are NodePool, EC2NodeClass, and NodeClaim.

10. Submit the following:

- Helm install commands used
- EC2NodeClass manifest
- NodePool manifest
- output of:
 - kubectl get nodepool
 - kubectl get nodeclaim
 - kubectl get nodes -o wide
- proof that the launched nodes are **t3a** and **Bottlerocket**

Here is the required design constraint for the manifests:

- **EC2NodeClass** must use amiFamily: Bottlerocket
- **NodePool** must allow only **t3a** instance types

- Use the **v1** Karpenter resources, not old Provisioner or AWSNodeTemplate objects, because current Karpenter uses NodePool and EC2NodeClass.

I can also turn this into a very short “student assignment” version with no technical explanation.